

# Assignment: Algorithm Implementation: Red-Black Tree

林柏翰

1113405021

國立澎湖科技大學

**Abstract**—本報告聚焦於進階資料結構中的自平衡二元搜尋樹：紅黑樹 (Red-Black Tree)。實驗實作了紅黑樹的節點插入與動態平衡調整演算法，透過旋轉與重新染色機制，驗證其在最壞情況下仍能維持  $O(\log N)$  的時間複雜度優勢。

**Index Terms**—Red-Black Tree, Self-Balancing BST, Time Complexity, Algorithms, Java.

## I. INTRODUCTION

在計算機科學中，普通的二元搜尋樹 (BST) 若因節點插入順序不佳，極易退化為類似鏈結串列的結構，導致操作的時間複雜度惡化至  $O(N)$ 。紅黑樹透過為每個節點增加顏色屬性 (紅色或黑色)，並嚴格遵守五大結構原則，強制整棵樹維持平衡，從而保證搜尋、插入與刪除操作的時間複雜度在最壞情況下皆能穩定維持在  $O(\log N)$ 。

## II. METHODOLOGY

紅黑樹在插入新節點時，首先完全依循普通 BST 的規則進行插入並將其預設為紅色，隨後檢查是否出現「紅紅相連 (Red-Red Violation)」等違規情況。若發生違規，則採用以下修復手段：

- **重新染色 (Recoloring):** 當父節點與叔叔節點皆為紅色時，將父、叔節點改為黑色，祖父節點改為紅色，並將問題向上遞迴處理。
- **旋轉 (Rotation):** 包括左旋 (Left Rotation) 與右旋 (Right Rotation)，用於調整樹的局部平衡與高度。

其樹高能精確控制在  $h \leq 2 \log(n + 1)$ ，因此保證了卓越的運算效率。

## III. EXPERIMENTAL RESULTS

本實驗使用 Java 進行實作。依據測試範例，程式依序插入 10, 20, 30, 15 四個節點，並在每次插入時自動執行平衡修正。終端機輸出的執行結果與時間複雜度如圖 ?? 所示。

## IV. CONCLUSION

本次實驗成功實作了紅黑樹的插入與平衡演算法。透過終端機輸出結果的驗證，確認了紅黑樹在動態資料維護中，能夠透過常數次的旋轉與染色修正，有效防止樹體退化，並切實達到  $O(\log N)$  的高效搜尋與維護效能。

```
--- Assignment 1: Red-Black Tree ---
Inserted: 10
Inserted: 20
Inserted: 30
Inserted: 15

[Time Complexity]
Insertion: O(log N)
Search: O(log N)
Deletion: O(log N)
PS C:\Users\User\Desktop\aa>
```

Fig. 1. 紅黑樹 (Red-Black Tree) 程式執行結果與時間複雜度。